Assignment 5

1.

Range()

The range() function **returns a sequence of numbers**, starting from **0** by default, and **increments by 1** (by default), and **stops before a specified number**.

Syntax:

range*(start, stop, step)*

Parameter Values

Parameter	Description
start	Optional. An integer number specifying at which position to start. Default is 0
stop	Required. An integer number specifying at which position to stop (not included).
step	Optional. An integer number specifying the incrementation. Default is 1

Subplot()

- The subplot() function **takes three arguments** that describes the layout of the figure.
- The layout is organized in **rows and columns**, which are represented by the **first and second argument**.
- The **third** argument represents the **index of the current plot**.

plt.subplot(1, 2, 1)

- #the figure has 1 row, 2 columns, and this plot is the first plot.

plt.subplot(1, 2, 2)

- #the figure has 1 row, 2 columns, and this plot is the second plot.
- Now, if we want a figure with 2 rows and 1 column (meaning that the two plots will be displayed on **top of each other instead of side-by-side**), we can write the syntax like this:

Legend()

Matplotlib.pyplot.legend()

- A legend is an **area describing the elements of the graph**. In the matplotlib library, there's a function called legend() which is used to Place a legend on the axes.
- The attribute **Loc** in legend() is used to specify the **location of the** legend. Default value of loc is loc="best" (upper left). The strings **'upper left', 'upper right', 'lower left', 'lower right'** place the legend at the corresponding corner of the axes/figure.
- The attribute **bbox_to_anchor=(x, y)** of legend() function is used to specify the **coordinates of the legend**, and the attribute **ncol** represents the **number of columns** that the legend has. It's default value **is 1.**
- The Following are some more attributes of function legend() :
  - **shadow**: [**None or bool**] Whether to draw a shadow behind the legend. It's Default value is None.
  - **markerscale**: [**None or int or float**] The relative **size of legend markers** compared with the originally drawn ones. The Default is None.
  - **numpoints**: [None or int] The **number of marker points in the legend** when creating a legend entry for a Line2D (line). The Default is None.
  - **fontsize**: The font size of the legend. If the value is **numeric** the size will be the absolute font size in points.
  - **facecolor**: [None or "inherit" or color] The legend's **background color**.
  - **edgecolor**: [None or "inherit" or color] The legend's **background patch** edge color.

Pandas DataFrame.columns

Create a Dataframe

- In this example, code imports the **Pandas and Matplotlib libraries creates a dictionary representing student data**, and uses it to create a Pandas DataFrame.
- The `head()` function displays the first five rows of the DataFrame.

len()

- In Python, the len() function is used to **determine the number of items in an object**.
- When the object is a **string**, len() returns the **number of characters in the string**.
- It can also be used with other data types like **lists, tuples, and dictionaries to get their sizes.**
- **Note:** Using len() on an empty object (like an empty list or string) will return 0.
- Usage with Different Data Types:
  - **Strings**: Returns the number of characters.
  - **Lists/Tuples**: Returns the number of elements.
  - **Dictionaries**: Returns the number of key-value pairs.

2.

5.2 Scatter plot: concept of scatter plot, set title, xlabel and ylabel

- Scatter plots are a fundamental tool in data visualization, providing a visual representation of the **relationship between two variables**.
- A scatter plot is a type of data visualization that displays **individual data points on a two-dimensional graph**. It uses **Cartesian coordinates** to display values for typically two variables for a set of data.
- The data points are represented **as dots**, where the position of each dot on the horizontal and vertical axis indicates **values for an individual data point**.
- Scatter plots are particularly useful for visualizing the relationship **between two continuous variables and identifying patterns, trends, correlations, and outliers** in the data.

Anatomy of a Scatter Plot

1. Axes and Data Points

- A typical scatter plot consists of two axes:
  - **X-Axis (Horizontal Axis):** Represents the **independent** variable.
  - **Y-Axis (Vertical Axis):** Represents the **dependent** variable.

2. Titles, Labels, and Legends

- **Title:** Provides a concise **description** of the plot's purpose or the data being visualized.
- **Axis Labels**: Indicate the **variables represented by the x and y axes.**
- **Legend**: If the plot contains **multiple datasets or different groups, a** legend explains what **each group represents**.

3. Gridlines and Annotations

- **Gridlines** improve **readability**, allowing viewers to estimate the values of points more accurately. **Annotations** can be added to **highlight specific points** or areas of interest in the scatter plot.

Creating a bar plot

The matplotlib API in Python provides the bar() function which can be used in MATLAB style use or as an object-oriented API. The syntax of the bar() function to be used with the axes is as follows:-

```
plt.bar(x, height, width, bottom, align)
```

Horizontal Bars

If you want the bars to be displayed horizontally instead of vertically, use the barh() function:

- Define the x-axis and corresponding y-axis values as lists.
- Plot them on canvas using **.plot()** function.
- Give a name to x-axis and y-axis using **.xlabel()** and **.ylabel()** functions.
- Give a title to your plot using **.title()** function.
- Finally, to view your plot, we use **.show()** function.

5.4 Histogram chart: Concepts of histogram hist(), set title, xlabel and ylabel

- A histogram is a graphical representation of the distribution of a dataset. It is a powerful tool for visualizing the **shape, spread, and central tendency** of a dataset.
- A histogram is a plot of the frequency distribution of numeric array by splitting it to small equal-sized bins.
- Unlike bar graphs, which are used for categorical data, histograms are designed for continuous data, grouping it into logical ranges or "bins."
- **"Bins"** is an optional parameter contains integer or sequence or strings

  number of intervals = "bins,"
- Range = Optional parameter represents upper and lower range of bins
- **Histograms** are commonly used in **data analysis, statistics, and machine learning to identify patterns, anomalies, and** trends in data.
- Pandas is a popular data manipulation and analysis library in Python. It provides a variety of functions and tools to work with **structured data, including reading, writing, filtering, cleaning, and transforming data**. Pandas also integrates well with other data visualization libraries such as **Matplotlib, Seaborn, and Plotly.**
- To create a histogram **from a Pandas DataFrame**, we first need to **extract the data** we want to plot. We can do this by selecting a **column from the DataFrame** using its name or **index**. Once we have the data, we can pass it to a **histogram function from** a visualization library to generate the plot.
- There are several **ways to create a histogram** from a Pandas DataFrame using different visualization libraries. For example, we can use the **"hist"** method from **Pandas**, the "**histogram**" function from **NumPy**. We can also **customize** the **appearance of the histogram by** changing the **colour, bins, title, axis labels**, and other properties.
- In Matplotlib, we can create a Histogram using the **hist method**.
- A histogram is a graph showing *frequency* distributions.
- It is a graph showing the number of observations within each given interval.

- In Matplotlib, we use the **hist()** function to create histograms.
- The hist() function will **use an array of numbers to create a histogram, the** array is sent into the function as an argument.

Syntax:

```
plt.hist(parameters)
```

Bar Color

The bar() and barh() take the keyword argument color to set the color of the bars:

```
plt.bar(x, y, color = "red")
```

Bar Width

The bar() takes the keyword argument width to set the width of the bars:

```
plt.bar(x, y, width = 0.1)
```

The default width value is 0.8

Note: For horizontal bars, use height instead of width.